

SCSI Pass-Through (spt) Tool

- [1. Overview](#)
 - [1.1. Robin's SCSI History](#)
- [2. Documentation](#)
- [3. Tool Location](#)
 - [3.1. Tool Invocation](#)
 - [3.1.1. Basic Command Format](#)
- [4. Scripts](#)
- [5. Simple Examples](#)
 - [5.1. Showing SCSI Devices](#)
 - [5.1.1. Linux Show SCSI Devices Example](#)
 - [5.2. SCSI Inquiry Examples](#)
 - [5.2.1. Inquiry ASCII Format](#)
 - [5.2.2. Inquiry JSON Format](#)
 - [5.2.2.1. Python JSON Parsing](#)
 - [5.2.2.2. Unpacking SCSI Data](#)
 - [5.3. Test Unit Ready](#)
 - [5.3.1. Test Unit Ready with Error Recovery](#)
- [6. Enhancing SCSI Automation](#)
- [7. Built-in \(shorthand\) SCSI Commands](#)
- [8. Advanced SCSI Examples](#)
 - [8.1. I/O Parameters](#)
 - [8.1.1. Write and Verify Examples](#)
 - [8.1.2. Extended Copy Example](#)
- [9. SCSI Debugging](#)
 - [9.1. Decoding SCSI Error Information](#)
- [10. SCSI Information](#)
 - [10.1. SCSI Statistics](#)
- [11. List Disks Script](#)
 - [11.1. Lsdisks Help Text](#)
 - [11.2. Example Brief Output Format:](#)
 - [11.3. Example Long Output Format:](#)
 - [11.4. Example JSON Output Format](#)
 - [11.5. Remote Example](#)

1. Overview

The [SCSI Pass-Through \(spt\) tool](#) is an open source tool, which I started developing in 2006.

What this tool provides is the ability to create any SCSI Command Descriptor Block (CDB), with appropriate parameters, controlling data direction, length, and timeouts, bypassing the host OS disk driver, and supporting built-in test functions. Constructing your own SCSI CDB's is permitted, in addition to built-in support for key SCSI commands (with more coming).

The initial spt version was single threaded, single device, much like the [data test \(dt\) tool](#), and in fact both tools share a fair amount of source code, including the SCSI library. 😊 Nonetheless, over the years, *spt* has been enhanced to be multi-threaded and extended with built-in SCSI command support for advanced testing, along with support for vendor unique (VU) extensions.

Note: VU extensions are *not* available in open source versions of dt or spt.

My hope when you review this tool, is that you'll see how it can make it easier and improve your SCSI testing, over many host OS SCSI tools used today!

Key spt features include: (common across all OS's)

- ability to send any SCSI CDB, controlling data direction, and parameters
- internal support for CDB's requiring varying parameters for looping:
 - read, write, verify, xcopy, unmap, etc
- multi-threaded for generating high load and stress
- supports both extended copy methods (xcopy and token based PT/WUT)
- copy/verify between disks, and support for mirror verification
- supports Storage Enclosure Services (SES) diagnostic pages
- output is available in ASCII or JSON (not all operations support JSON yet)
- ability to decode SCSI status, sense keys, and asc/ascq codes
- ability to verify data returned automatically (much faster than scripts)
- ability to repeat commands while waiting for specific SCSI status returns
- Python script named *lsdisks.py* wrapper around spt for displaying SES/disk information
- on Linux, spt is built as a shared object (.so), using Python CTYPES API to call directly into spt! 😊
- *spt* is feature rich, mature, and provides a unique set of features most SCSI tools only dream about!

Please Note: Unlike *dt*, *spt* lacks dynamic jobs control, I/O monitoring, triggers, and others. The hope has always been to integrate *spt* into *dt* as an I/O behavior, so all the *dt* features would be available. This may still happen in the future, we'll see if there's sufficient demand to justify the effort.

UPDATE: Not likely to happen now that I'm retired with limited hardware available. 😞

1.1. Robin's SCSI History

My start with SCSI started in 1986, upon working on Sun Microsystems SunOS. This style SCSI is known as parallel SCSI (pSCSI) today and had a limited number of target devices and the target ID's even provided a bus priority, with the host initiator normally at the highest ID (7 on an 8-bit SCSI bus).

I learned much from studying the SunOS SCSI drivers and the associated *format* utility, which I updated to support extended SCSI commands beyond normal Format and Defect management. I then wrote a

simple SCSI I/O stack for System V, from a WD HBA to disk and tape drives for a embedded typesetter product.

Later when I joined DEC, I did SCSI device qualification and driver development, wrote a *cdr* tool with CD audio support, then later wrote the SCSI CAM Utility (*scu*) starting around 1990. *Scu* ships with DEC's Tru64 Unix OS, for field service SCSI device maintenance and FW updates. Later I ported *Scu* to most Unix OS's for the Data Product Group for testing and later ported *Scu* to Windows. Thereafter I enhanced the CAM configuration driver, developed a tape library driver, and a special IOCTL interface for handling user level SCSI requests, used by *Scu* as well (re-use of kernel code in user level tools).

From the original CCS/SCSI-2 ANSI standards to the current SCSI Primary Commands (SPC-5) and other SCSI specs, trying to keep up with ever changing SCSI improvements is a full-time job. SCSI is wrapped by many protocols, from iSCSI to FC, FCoE, to SAS, but the basic SCSI CDB's are contained within each.

Even though NVME appears to be the future, which will **not** wrap SCSI packets, the SCSI protocol will likely live on long after I'm retired and gone!

2. Documentation

Most of the documentation is in the "*spt help*" text and in these TOI's: (sorry, never wrote a users' manual)

- [Small Computer System Interface \(SCSI\)](#)
- spt-TOI-2017.pptx – Transfer of Information (TOI) 2017
- spt-TOI-2019.pptx – Transfer of Information (TOI) 2019

3. Tool Location

My tools are available here: <https://github.com/RobinTMiller>

For help type: # *spt help*

3.1. Tool Invocation

There are multiple ways to invoke spt, including:

- Command Line Interface (CLI) - most tools support this, of course
- Script Files - these can be nested five (5) levels deep
- Pipe Interface - send commands and receive output via pipes
- Interactive - enter interactive mode when no command is provided
- Python API - shared library supporting Python CYPES API

3.1.1. Basic Command Format

spt uses similar parsing to *dt*, that is keyword=value pairs and enable/disable= for flags:

```
# spt dsf=/dev/... cdb=hex,hex... [ dir={none|read|write} [ length=value ] ...
```

The device special file (**dsf=**) is required, along with a CDB to send. (see "*spt help*")

Beware: While many of us use /dev/mapper or /dev/sd* devices for issuing SCSI commands, my recommendation is to use the associated "sg" device on Linux, since drivers can and do occasionally

"steal SCSI state" information, such as SCSI status, request sense, etc. *spt* supports an **enable=mapscsi** to do this automatically (I may change this to the default). Even system daemons such as *smartd* and multipathing, send SCSI commands at intervals that may interfere with your SCSI testing.

Also Note: The Linux block "sg" (/dev/bsg) interface is **not** supported today (maybe in the future, has not been high priority so far).

4. Scripts

You can add spt commands to a script file to perform a set of advanced workloads:

- scripts have a default extension of **.spt**
- environment variables can be expanded in the form of \${VAR}
- environment variables can be set using this syntax:
 - `$VAR="text"` - Always set variable.
 - `$VAR=${VAR:-"text"}` - Only set variable, if **not** already set.
 - `$VAR=${VAR:?message}` - Report a message and exit w/failure.
- add **"enable=scriptverify"** to trace script execution, otherwise it's silent.
- use the **"script=FILE[.spt]"** command to execute the script file.

While the current script support is limited, it's sufficient for simple dt scripts. Extended scripts are expected to be written in other languages such as Bash, Perl, Python, etc, since these languages provide all the looping and variable substitution constructs desired by advanced script writers.

5. Simple Examples

The **"spt help"** has a long set of example commands. Please know these key points:

- The **"dsf="** option is required for a single device for the Device Special File (dsf):
 - e.g.: `/dev/sdbx`, `/dev/sg76`, `/dev/dm-7`, or `/dev/mapper/mpathae`
- The environment variable SPT_DEVICE can be defined for the spt device.
 - e.g.: `# export SPT_DEVICE=/dev/mapper/mpathae`
- spt does **not** display statistics automatically (roll your own, see SCSI section below)

Here's a few examples from the spt help text:

```

Builtin Support Examples:
Inquiry Information: (human readable)
# spt inquiry logprefix=
Read Capacity(10): (for older SCSI devices or USB)
# spt readcapacity10
Read Capacity(16): (shows thin provisioning)
# spt readcapacity16 ofmt=json
Report LUNs:
# spt cdb=a0 enable=encode,decode disable=verbose
Report Target Group Support:
# spt cdb=a3 0a enable=encode,decode disable=verbose
Write and Read/Compare IOT Pattern: (32k, all blocks)
# spt cdb=8a dir=write length=32k enable=compare,recovery,sense starting=0 ptype=iot
Read and Compare IOT Pattern: (32k, all blocks)
# spt cdb=88 dir=read length=32k enable=compare,recovery,sense starting=0 ptype=iot
Write and Read/Compare IOT Pattern w/Immediate Read-After-Write: (64k, 1g data)
# spt cdb=8a starting=0 bs=64k limit=1g ptype=iot enable=raw emit=default
Write Same: (all blocks)
# spt cdb=93 starting=0 dir=write length=4k blocks=4m/b
Write Same w/unmap: (all blocks)
# spt cdb=93 08 starting=0 dir=write length=512 blocks=4m/b
Unmap all blocks: (incrementing blocks per range)
# spt cdb=42 starting=0 ranges=64 min=8 max=128 incr=8
Get LBA status: (reports mapped/deallocated blocks)
# spt cdb=9e 12 starting=0
Extended Copy Operation: (non-token based, used by VMware)
# spt cdb=83 src=${SRC} starting=0 dst=${DST} starting=0 enable=compare,recovery,sense
Extended Copy operation: (token based, used by Microsoft, aka ODX)
# spt cdb=83 11 src=${SRC} starting=0 dst=${DST} starting=0 enable=compare,recovery,sense
Zero ROD Token: (10 slices, all blocks, space allocation needs enabled)
# spt cdb=83 11 dsf=${DSF} starting=0 enable=zerorod slices=10 enable=recovery,sense
Copy/Verify Source to Destination Device: (uses read/write operations)
# spt tomode=copy length=32k dsf=${SRC} starting=0 dsfl=${DST} starting=0 enable=compare,recovery,sense
Write Source and Verify with Mirror Device: (10 threads for higher performance)
# spt tomode=mirror length=32k dsf=${SRC} starting=0 dsfl=${DST} starting=0 enable=compare slices=10

Environment Variables:
# export SPT_DEVICE="/dev/sdi"
# export SPT_SHOW_FIELDS="dtype,vid,pid,did,tport,paths"
# export SPT_EMIT_STATUS="status: %status, SCSI Status: %scsi_status, Sense Code: %sense_code, Sense Key: %sense_key, Ascq: %ascq, Resid: %resid"

--> Date: April 26th, 2019, version: 3.23, Author: Robin T. Miller <--
[root@rtppqa-rhel001 Scripts]#

```

5.1. Showing SCSI Devices

Every Host OS has its own tools for showing devices attached to the system. On Linux for SCSI, the most popular are *lsscsi* and *multipath* for DM-MP (MPIO) devices. Unlike *lsscsi* which displays cached SCSI information from the */sys* file system, *spt* requests its' information directly from the SCSI device. Please keep this in mind, since sending SCSI commands to be misbehaving or disconnected devices, WILL cause *spt* to hang even with its' default 60 second timeout. 😞

While OS tools are useful, oftentimes those working with SCSI want more SCSI specific information, and/or may desire an output format other than ASCII to avoid "screen scraping" and simplify parsing.

Therefore, for Linux and Windows today, I've added a "show device" command to *spt* with options for filtering and various output formats.

Show Devices Filters:

device(s)=string - The device path(s).
device_type(s)={value|string},... - The device type. (or **dtype=**)
exclude=string - The exclude path(s). (for bad or decommissioned drives)
product=string - The product name. (or **pid=**)
vendor=string - The vendor name. (or **vid=**)
revision=string - The revision level. (or **rev=**)
fw_version=string - The firmware version. (or **fwver=**)
serial=string - The serial number.
show-fields=string - Show devices brief fields. (or **sflds=**).
show-format=string - Show devices format control. (or **sfmt=**).
show-path=string,... - Show devices using path. (or **spath=**).

5.1.1. Linux Show SCSI Devices Example

Here's what the default output looks like:

SCSI PASS-THROUGH (SPT) TOOL

```
[root@rtppqa-rhel001 ~]# spt show devices
Device Type  Vendor      Product      Revision      Serial Number      Device Paths
-----
Direct       VMware     Virtual disk  2.0           <not available>    /dev/sda /dev/sg0
Direct       Nimble     Server        1.0           1d83ceal03fcacba6c9ce900ed956192 /dev/sdb /dev/sg2 /dev/sde /dev/sg5 /dev/sdh /dev/sg8 /dev/sdk /dev/sg11 /dev/sdn /dev/sg14 /dev/sdq /dev/sg17 /
dev/sdt /dev/sg20 /dev/sdw /dev/sg23 /dev/sdz /dev/sg26 /dev/sdac /dev/sg29 /dev/sdaf /dev/sg32 /dev/sda1 /dev/sg35 /dev/mapper/mpathac
Direct       Nimble     Server        1.0           aa6960b38d0cf0946c9ce900ed956192 /dev/sdc /dev/sg3 /dev/sdf /dev/sg6 /dev/sd1 /dev/sg9 /dev/sd1 /dev/sg12 /dev/sdo /dev/sg15 /dev/sdr /dev/sg18 /
dev/sdu /dev/sg21 /dev/sdx /dev/sg24 /dev/sdaa /dev/sg27 /dev/sdad /dev/sg30 /dev/sdag /dev/sg33 /dev/sda1 /dev/sg36 /dev/mapper/mpathad
Direct       Nimble     Server        1.0           7b465faa2e1855ac6c9ce900ed956192 /dev/sdd /dev/sg4 /dev/sdg /dev/sg7 /dev/sd1 /dev/sg10 /dev/sdm /dev/sg13 /dev/sdp /dev/sg16 /dev/sds /dev/sg19 /
dev/sdv /dev/sg22 /dev/sdy /dev/sg25 /dev/sdab /dev/sg28 /dev/sdae /dev/sg31 /dev/sdah /dev/sg34 /dev/sdak /dev/sg37 /dev/mapper/mpathae
CD/DVD      NECVMwar  VMware IDE CDR10  1.00          <not available>    /dev/sg1
[root@rtppqa-rhel001 ~]#
```

Since Linux has many SCSI device names, the last field can be rather long. Below, you'll see the *lsdisks* script which provides an alternate format.

Here's an example of JSON output for a single device:

```
[root@rtppqa-rhel001 ~]# spt show edt devices=/dev/mapper/mpathae ofmt=json
{
  "SCSI Devices": {
    "Device List": [
      {
        "Peripheral Device Type": 0,
        "Peripheral Device Type Description": "Direct Access Device",
        "Vendor Identification": "Nimble ",
        "Product Identification": "Server ",
        "Firmware Revision Level": "1.0 ",
        "Full Firmware Version": "<not available>",
        "Product Serial Number": "7b465faa2e1855ac6c9ce900ed956192",
        "Device World Wide Name": "0x7b465faa2e1855ac6c9ce900ed956192",
        "Device Target Port": "<not available>",
        "Array SW Version": "5.1",
        "Target Type": "Group Scoped",
        "Sync Replication": false,
        "Device Paths": "\dev\mapper\mpathae",
        "Path Types": [
          {
            "DMMP Device": "\dev\mapper\mpathae",
            "SCSI Nexus": "[3:0:10:2]"
          }
        ]
      }
    ],
    "JSON Status": 0
  }
}
[root@rtppqa-rhel001 ~]#
```

The *lsdisks* script can be used as a reference for parsing JSON in Python.

Here's another example to find all the Nimble SCSI devices and create a list of same:

```
# export DISKS=$(spt show devices spaths=/dev/mapper/mpath sfmt="%paths' vid=Nimble)
# echo $DISKS
/dev/mapper/mpathac /dev/mapper/mpathad /dev/mapper/mpathae
#
```

There are many other formats, including a roll-your-own output format and configuring the summary fields, but alas I'll save covering these for a future deep dive!

5.2. SCSI Inquiry Examples

5.2.1. Inquiry ASCII Format

Notice the Nimble specific information that's been decoded, since spt was enhanced:

SCSI PASS-THROUGH (SPT) TOOL

```
[root@rtppqa-rhel001 ~]# export SPT_DEVICE=/dev/mapper/mpathae
[root@rtppqa-rhel001 ~]# spt inquiry logprefix=

Inquiry Information:

    Peripheral Device Type: 0 (Direct Access Device)
    Peripheral Qualifier: 0 (Peripheral Device Connected)
    Logical Unit Conglomerate: False
    Removable Media: False
    ANSI Version: 5 (Complies to ANSI INCITS 408-2005, SPC-3)
    Response Data Format: 2
    Historical Support: True
    Normal ACA Support: True
    Additional Length: 63 (0x3f)
    Supports Protection Information: False
    Third Party Copy Support: True
    Target Port Group Support: 0x1 (implicit ALUA)
    Storage Controller Components: False
    Multiple SCSI Ports: True
    Embedded Enclosure Services: False
    Command Queuing Support: True
    Vendor Identification: Nimble
    Product Identification: Server
    Firmware Revision Level: 1.0
    Array Software Version: 5.1
    Target Type: 1 (Group Scoped)
    Synchronous Replication: False
    Vendor Data Length: 11 (0xb)
    Vendor Specific Data: 00 00 00 60 09 60 08 c0 03 00 03

[root@rtppqa-rhel001 ~]# █
```

5.2.2. Inquiry JSON Format

For scripting languages, I prefer JSON format versus "screen scraping", thus:

[illegible]

You'll notice the hex data is also reported, since SCSI engineers often wish to see the raw data bytes.

5.2.2.1. Python JSON Parsing

Snippet from *lsdisks* script:

```

message = "Get Inquiry Information"
command = "spt dsf={dsf} inquiry output-format=json emit=".format(dsf=sdsf)
pdata = self._run_command(command=command, message=message, logger=self._logger, shell=False)
inquiry = json.loads(pdata['stdout'])
device['Device Type Description'] = inquiry['Inquiry']['Peripheral Device Type Description']
device['Product Identification'] = inquiry['Inquiry']['Product Identification'].strip()
device['Vendor Identification'] = inquiry['Inquiry']['Vendor Identification'].strip()
# The version is returned for ATA drives, so use it.
if device.get('Revision Level') is None:
    device['Revision Level'] = inquiry['Inquiry']['Firmware Revision Level'].strip()
device['Array Software Version'] = inquiry['Inquiry'].get('Array Software Version', "")
device['Target Type'] = inquiry['Inquiry'].get('Target Type')
device['Target Type Description'] = inquiry['Inquiry'].get('Target Type Description')
device['Synchronous Replication'] = inquiry['Inquiry'].get('Synchronous Replication')

```

5.2.2.2. Unpacking SCSI Data

There are many SCSI commands, and *spt* does not support them all. Therefore, to prevent scripts from decoding hex or raw data bytes, there's an unpack option to help you define how *spt* can unpack data for you. Of course, you still need to know the data format returned for each SCSI command, usually taken from the SCSI or vendor specs. Please know, today we do **not** have a Nimble SCSI spec, one needs to reference the product include files.

Unpack Data Options:

unpack=string - The unpack format string.

unpack_fmt={dec,hex} - The unpack data format. (Default: dec)

Where unpack format string is:

C[HAR]:[index]:length - Character of length.

F[IELD]:[index]:[start]:length - Extract bit field.

O[FFSET]:index - Set the buffer offset.

B[YTE]:[index] - Decode byte (8 bit) value.

S[HORT]:[index] - Decode short (16 bit) value.

W[ORD]:[index] - Decode word (32 bit) values.

L[ONG]:[index] - Decode long (64 bit) values.

Inquiry Unpack Examples:

```

# spt dsf=/dev/mapper/mpathac inquiry disable=decode \
unpack='Device Type: %BYTE, Vendor: %CHAR:8:8, Product: %C::16, Revision: %C::4\n'

```

Device Type: 0, Vendor: Nimble , Product: Server , Revision: 1.0

OR Create your own JSON: (multiple unpack's permitted)

```

# spt dsf=/dev/mapper/mpathac inquiry disable=decode \
unpack='{ "Device Type": %BYTE, "Vendor": "%C:8:8", '\
unpack=' "Product": "%C::16", "Revision": "%C::4" }\n'

{ "Device Type": 0, "Vendor": "Nimble ", "Product": "Server ", "Revision": "1.0 " }

```

5.3. Test Unit Ready

Often the next SCSI command issued after an Inquiry, is often a SCSI Test Unit Ready (TUR) to see if the device is ready to accept commands:

```
# spt showopcodes test
```

Opcode	Subcode	Direction	Encode	Decode	Default	Opcode Name
0x00	0x00	none	no	no	0	Test Unit Ready

spt cdb=0

Since spt does **not** provide built-in support for all SCSI commands, for those missing, you need to specify the SCSI operation code (opcode).

The commands used above, **showopcodes** allows you to search for a SCSI command, **cdb=0** executes TUR.

5.3.1. Test Unit Ready with Error Recovery

Then to force an error, **lun_reset** sends a LUN Reset TMF, with another TUR with error recovery enabled. Pretty cool, eh?

```
# spt op=lun_reset ; spt cdb=0 enable=recovery
spt (j:1 t:1): Warning: Error occurred on Sun Jul 28 12:00:42 2019
spt (j:1 t:1): Test Unit Ready failed on device /dev/mapper/mpathae
spt (j:1 t:1): SCSI Status = 0x2 (SCSI_CHECK_CONDITION)
spt (j:1 t:1): Host Status = 0 (DID_OK), Driver Status = 0x8 (DRIVER_SENSE)
spt (j:1 t:1): Sense Key = 6 = UNIT ATTENTION, Sense Code/Qualifier = (0x29, 0x3) - Bus device
reset function occurred
spt (j:1 t:1): Warning: Retrying Test Unit Ready after 2 second delay, retry #1...
#
```

OR, you can instruct spt through a its' built-in test option, to wait for a specific SCSI status with delays and retries: (even cooler! 😊)

```
# spt op=lun_reset ; spt cdb=0 status=good retry=100 msleep=100 enable=wait
```

Note: You won't see the errors or retries unless you enable=debug, unless all retries fail (of course).

For shits and grins, let's force retries to be exhausted so we can see the resulting error messages:

```
# spt op=lun_reset ; spt cdb=0 status=cc retry=100 msleep=100 enable=wait
spt (j:1 t:1): Result for Test Unit Ready
spt (j:1 t:1): Expected:
spt (j:1 t:1):   SCSI Status: 02h = SCSI_CHECK_CONDITION
spt (j:1 t:1):   Sense Key: 00h = NO SENSE
spt (j:1 t:1):   asc/ascq: 00h/00h = No additional sense information
spt (j:1 t:1): Actual:
spt (j:1 t:1):   SCSI Status: 00h = SCSI_GOOD
spt (j:1 t:1):   Sense Key: 00h = NO SENSE
spt (j:1 t:1):   asc/ascq: 00h/00h = No additional sense information
spt (j:1 t:1): ERROR: Error number 1 occurred on Sun Jul 28 12:19:00 2019
spt (j:1 t:1): ERROR: Retry limit of 100 reached for SCSI opcode 0x0 (Test Unit Ready)
# echo $?
255
#
```

BEWARE: The LUN Reset likely re-enumerated your SCSI "sd" and "sg" devices, a **not** so wonderful Linux behavior! 😞

Ok, I'd need a full presentation to cover all the test specific options with examples, but the point is by moving such things into *spt*, the tool can do most of the heavy lifting, and do it much faster than script wrappers! And because *spt* has a roll your own emit status string, your automation can parse resulting error information much easier, without "screen scraping"! (I'm not a fan of)

7. Built-in (shorthand) SCSI Commands

spt was originally written to send any CDB without internal support for encoding and/or decoding SCSI commands.

But eventually, it became clear that shorthand commands make it more user friendly, towards the *scu* tool mentioned.

First various I/O type CDB's were enhanced, mainly since they require CDB updates when looping across a wide range of data.

Next, a set of SES and these SCSI commands were enhanced, since they were used in automation at my last company:

Shorthand Commands: (with more to come)

- **inquiry {page=value}** - Show Inquiry or specific page.
- **logsense {page=value}** - Show Log pages supported or page.
- **zerolog {page=value}** - Zero all Log pages or specific page.
- **readcapacity10** - Show disk capacity (10 byte CDB).
- **readcapacity16** - Show disk capacity (16 byte CDB).
- **requestsense** - Show request sense information.
- **rtpg** - Report target port groups.

Storage Enclosure Services (SES) Specific Options:

- **rcvdiagpage=value** - Issue a receive diagnostic command.
- **senddiag page=value** - Issue a send diagnostic command.
- **showhelp** - Show enclosure help text diagnostic page.
- **ses {clear|set}={devoff|fail/fault|ident/locate|unlock}** - Modify SES control elements.

As you may know, some SCSI opcodes support numerous pages as shown above, and not all pages are decoded, but rather the ones requested to date.

8. Advanced SCSI Examples

As mentioned above, a set of options were added to allow I/O type commands across a set of blocks:

I/O Options:

- **lba=value** - The logical block address.
- **bs=value** - The number of bytes per request.
- **blocks=value** - The number of blocks per request.

- **limit=value** - The data limit to transfer (bytes).
- **ptype=string** - The pattern type (only 'iot' now).
- **ending=value** - The ending logical block address.
- **starting=value** - The starting logical block address.
- **slices=value** - The slices to divide capacity between.
- **step=value** - The bytes to step after each request.

Those of you familiar with my *dt* tool, will recognize most of these options, while several are specific to SCSI.

8.1. I/O Parameters

The I/O parameters are used in conjunction with the CDB operation, here are SCSI write opcodes:

#	spt	show	opcodes	write				
	Opcode	Subcode	Direction	Encode	Decode	Default	Opcode Name	
	0x3b	0x00	none	no	no	0	Write Buffer	
	0x0a	0x00	write	yes	no	0	Write(6)	
	0x2a	0x00	write	yes	no	0	Write(10)	
	0x2e	0x00	write	yes	no	0	Write and Verify	
	0x3f	0x00	none	no	no	0	Write Long	
	0x41	0x00	write	yes	no	65535	Write Same	
	0x83	0x11	write	yes	no	131072	XCOPY - Write Using Token	
	0x8a	0x00	write	yes	no	0	Write(16)	
	0x8e	0x00	write	yes	no	0	Write and Verify(16)	
	0x93	0x00	write	yes	no	65536	Write Same(16)	
	0x89	0x00	write	yes	no	1	Compare and Write(16)	

FYI: Eventually I'll enhance the parsing to lookup opcodes by names, just need to find/make the time!

8.1.1. Write and Verify Examples

Here's a simple single threaded Write and Verify command line:

```
# spt cdb=0x8e bs=256k starting=0 limit=1g ptype=iot emit=default logprefix=
    Thread: 1
    Device Name: /dev/mapper/mpathae
    Device Info: Block Length=512, Capacity=61440000
    Block Range: 0 - 2097152
    SCSI Name: Write and Verify(16)
    SCSI CDB: 8e 00 00 00 00 00 00 1f fe 00 00 00 02 00 00 00
    Data Direction: write
    Data Length: 262144
    Exit Status: 0 = SUCCESS
    Host Status: 0 = DID_OK
    Driver Status: 0 = DRIVER_OK
    SCSI Status: 0 = SCSI_GOOD
    Sense Code: 0 = NO CODE
    Sense Key: 0 = NO SENSE
    asc/asq: (00, 00) = No additional sense information
    Bytes Transferred: 262144 (data bytes transferred)
    Residual: 0 (bytes not transferred)
    Iterations: 1
    Total Bytes: 1073741824
    Total Blocks: 2097152
    Total Operations: 4097
    Sense Data: 00 00 00 00 00 00 00 00
    Elapsed Time: 00m03.94s
    Starting Time: Mon Jul 29 11:08:43 2019
    Ending Time: Mon Jul 29 11:08:47 2019
#
```

Remember unlike *dt*, if a data pattern is **not** specified, then the pattern written is zeros! (unmaps Nimble blocks)

Also know, that *spt* does **not** do error recovery by default, since as a test tool usually want the error status.

Therefore, add **enable=recovery** if you like certain SCSI errors such as SCSI_BUSY or Unit Attention (UA) to be retried automatically. Otherwise errors such as this one will cause a failure, even though the UA is retryable: (even I *the author*, often gets bit by this lack of automatic error recovery 🙄)

```
# spt cdb=0x8e bs=256k starting=0 slices=10 ptype=iot enable=Debug
spt (:0 t:0): Device: /dev/mapper/mpathae, Device Size: 512 bytes, Capacity: 61440000 blocks (thread 0)
spt (:0 t:0): Device: /dev/mapper/mpathae, Starting lba=0, ending lba=61439999, block limit=61440000 (thread 0)
spt (:1 t:3): Device: /dev/mapper/mpathae, Starting lba=12288000, ending lba=18431999, block limit=61440000 (thread 3)
spt (:1 t:5): Device: /dev/mapper/mpathae, Starting lba=24576000, ending lba=30719999, block limit=61440000 (thread 5)
spt (:1 t:2): Device: /dev/mapper/mpathae, Starting lba=6144000, ending lba=12287999, block limit=61440000 (thread 2)
spt (:1 t:3): ERROR: Error number 1 occurred on Mon Jul 29 11:28:59 2019
spt (:1 t:3): write and verify(16) failed on device /dev/mapper/mpathae (thread 3)
spt (:1 t:3): SCSI CDB: 8e 00 00 00 00 00 00 bb 80 00 00 00 02 00 00 00, dir=write, length=262144
spt (:1 t:3): SCSI Status = 0x2 (SCSI_CHECK_CONDITION)
spt (:1 t:3): Host Status = 0 (DID_OK), Driver Status = 0x8 (DRIVER_SENSE)
spt (:1 t:3): Sense Key = 6 = UNIT ATTENTION, Sense Code/Qualifier = (0x38, 0x7) - Thin provisioning soft threshold reached
spt (:1 t:3):
spt (:1 t:3): Request Sense Data: (sense length 24 bytes)
spt (:1 t:3):
spt (:1 t:3): Error Code: 0x70 = Current Error
spt (:1 t:3): Information Field Valid: Yes
spt (:1 t:3): Sense Key: 0x6 = UNIT ATTENTION
spt (:1 t:3): Information Field: 0 (0)
spt (:1 t:3): Additional Sense Length: 16 (0x10)
spt (:1 t:3): Command Specific Information: 0 (0)
spt (:1 t:3): Additional Sense Code/Qualifier: (0x38, 0x7) - Thin provisioning soft threshold reached
spt (:1 t:3): Field Replaceable Unit code: 0
spt (:1 t:3): Sense Key Specific Bytes: 00 00 00
spt (:1 t:3): Additional Sense Bytes: 00 00 00 00 00 00
spt (:1 t:3):
```

And here's an example of using slices (multiple threads) across the entire capacity of the drive:

```
# spt cdb=0x8e bs=256k starting=0 slices=10 ptype=iot enable=Debug
spt (:0 t:0): Device: /dev/mapper/mpathae, Device Size: 512 bytes, Capacity: 61440000 blocks (thread 0)
spt (:0 t:0): Device: /dev/mapper/mpathae, Starting lba=0, ending lba=61439999, block limit=61440000 (thread 0)
spt (:1 t:1): Device: /dev/mapper/mpathae, Starting lba=0, ending lba=6143999, block limit=61440000 (thread 1)
spt (:1 t:3): Device: /dev/mapper/mpathae, Starting lba=12288000, ending lba=18431999, block limit=61440000 (thread 3)
spt (:1 t:5): Device: /dev/mapper/mpathae, Starting lba=24576000, ending lba=30719999, block limit=61440000 (thread 5)
spt (:1 t:7): Device: /dev/mapper/mpathae, Starting lba=36864000, ending lba=43007999, block limit=61440000 (thread 7)
spt (:1 t:2): Device: /dev/mapper/mpathae, Starting lba=6144000, ending lba=12287999, block limit=61440000 (thread 2)
spt (:1 t:6): Device: /dev/mapper/mpathae, Starting lba=30720000, ending lba=36863999, block limit=61440000 (thread 6)
spt (:1 t:8): Device: /dev/mapper/mpathae, Starting lba=43008000, ending lba=49151999, block limit=61440000 (thread 8)
spt (:1 t:4): Device: /dev/mapper/mpathae, Starting lba=18432000, ending lba=24575999, block limit=61440000 (thread 4)
spt (:1 t:10): Device: /dev/mapper/mpathae, Starting lba=55296000, ending lba=61439999, block limit=61440000 (thread 10)
spt (:1 t:9): Device: /dev/mapper/mpathae, Starting lba=49152000, ending lba=55295999, block limit=61440000 (thread 9)
#
```

You can imagine the load and stress generated when doing multi-threaded SCSI operations across a larger set of disks!

8.1.2. Extended Copy Example

There are two styles of extended copy operations, the original XCOPY and the token based XCOPY.

Here is an example of doing an image mode copy between two disks:

SCSI PASS-THROUGH (SPT) TOOL

```
# spt src=/dev/mapper/mpathac dst=/dev/mapper/mpathad cdb=83,11 starting=0 enable=Debug,image emit=multi logprefix=
Device: /dev/mapper/mpathad, Device Size: 512 bytes, Capacity: 40960000 blocks (thread 0)
Device: /dev/mapper/mpathac, Device Size: 512 bytes, Capacity: 20480000 blocks (thread 0)
WARNING: The device capacity is different between the selected devices!
Source Device: /dev/mapper/mpathac, Capacity: 20480000 blocks
Destination Device: /dev/mapper/mpathad, Capacity: 40960000 blocks
Setting both devices to the smallest capacity to ensure the same block ranges!
Device: /dev/mapper/mpathad, starting lba=0, ending lba=20479999, block limit=20480000 (thread 1)
Device: /dev/mapper/mpathac, starting lba=0, ending lba=20479999, block limit=20480000 (thread 1)

Thread: 1
Source Device: /dev/mapper/mpathac
Source Info: Block Length=512, Capacity=20480000
Block Range: 0 - 20480000
Destination Device: /dev/mapper/mpathad
Destination Info: Block Length=512, Capacity=20480000
Block Range: 0 - 20480000
SCSI Name: XCOPY - write using Token
SCSI CDB: 83 11 00 00 00 00 01 8b 46 03 00 00 02 28 00 00
Data Direction: write
Data Length: 512
Exit Status: 0 = SUCCESS
Host Status: 0 = DID_OK
Driver Status: 0 = DRIVER_OK
SCSI Status: 0 = SCSI_GOOD
Sense Code: 0 = NO CODE
Sense Key: 0 = NO SENSE
asc/asq: (00, 00) = No additional sense information
Bytes Transferred: 32 (data bytes transferred)
Residual: 773 (bytes not transferred)
Iterations: 1
Total Bytes: 86664
Total Blocks: 20480000
Total Operations: 632
Sense Data: 00 00 00 00 00 00 00 00
Elapsed Time: 00m11.64s
Starting Time: Mon Jul 29 12:01:53 2019
Ending Time: Mon Jul 29 12:02:05 2019

#
```

For proper testing, the source disk should be pre-populated with valid data, then destination verified via:

```
# spt dsf=/dev/mapper/mpathad cdb=88 starting=0 bs=256k pattern=iot limit=20480000b
```

OR you can simply verify the source and destination disks via this command:

```
# spt iomode=verify length=256k dsf=${SRC} starting=0 dsf1=${DST} starting=0 emit=multi enable=compare,recovery,sense logprefix=
WARNING: The device capacity is different between the selected devices!
Source Device: /dev/mapper/mpathac, Capacity: 20480000 blocks
Destination Device: /dev/mapper/mpathad, Capacity: 61440000 blocks

Thread: 1
Source Device: /dev/mapper/mpathad
Source Info: Block Length=512, Capacity=61440000
Block Range: 0 - 61440000
Destination Device: /dev/mapper/mpathac
Destination Info: Block Length=512, Capacity=20480000
Block Range: 0 - 20480000
SCSI Name: Read(16)
SCSI CDB: 88 00 00 00 00 00 01 38 7e 00 00 00 02 00 00 00
Data Direction: read
Data Length: 262144
Exit Status: 0 = SUCCESS
Host Status: 0 = DID_OK
Driver Status: 0 = DRIVER_OK
SCSI Status: 0 = SCSI_GOOD
Sense Code: 0 = NO CODE
Sense Key: 0 = NO SENSE
asc/asq: (00, 00) = No additional sense information
Bytes Transferred: 262144 (data bytes transferred)
Residual: 0 (bytes not transferred)
Iterations: 1
Total Bytes: 20971520000
Total Blocks: 40960000
Total Operations: 80002
Sense Data: 00 00 00 00 00 00 00 00
Elapsed Time: 00m29.06s
Starting Time: Mon Jul 29 12:23:47 2019
Ending Time: Mon Jul 29 12:24:16 2019

#
```

I often find *scu* convenient for dumping LBA's via:

SCSI PASS-THROUGH (SPT) TOOL

```
# scu -f /dev/mapper/mpathad dump media lba 0
Reading 1 block (0 - 0) on /dev/mapper/mpathad (Server)...

Record Number 1 (Starting Block 0)

00000000 00 00 00 00 01 01 01 02 02 02 02 03 03 03 04 04 04 05 05 05 06 06 06 07 07 07 08 08 08 08 09 09 09
00000040 0a 0a 0a 0a 0b 0b 0b 0c 0c 0c 0c 0d 0d 0d 0e 0e 0e 0f 0f 0f 10 10 10 11 11 11 12 12 12 13 13 13
00000080 14 14 14 14 15 15 15 16 16 16 17 17 17 18 18 18 19 19 19 1a 1a 1a 1b 1b 1b 1c 1c 1c 1d 1d 1d
00000120 1e 1e 1e 1f 1f 1f 20 20 20 21 21 21 22 22 22 23 23 23 24 24 24 25 25 25 26 26 26 27 27 27
00000160 28 28 28 28 29 29 29 2a 2a 2a 2b 2b 2b 2c 2c 2c 2d 2d 2d 2e 2e 2e 2f 2f 2f 30 30 30 31 31 31
00000200 32 32 32 32 33 33 33 34 34 34 35 35 35 36 36 36 37 37 37 38 38 38 39 39 39 3a 3a 3a 3b 3b 3b
00000240 3c 3c 3c 3c 3d 3d 3d 3e 3e 3e 3f 3f 3f 40 40 40 41 41 41 42 42 42 43 43 43 44 44 44 45 45 45
00000280 46 46 46 46 47 47 47 48 48 48 49 49 49 4a 4a 4a 4b 4b 4b 4c 4c 4c 4d 4d 4d 4e 4e 4e 4f 4f 4f
00000320 50 50 50 51 51 51 52 52 52 53 53 53 54 54 54 55 55 55 56 56 56 57 57 57 58 58 58 59 59 59
00000360 5a 5a 5a 5b 5b 5b 5c 5c 5c 5d 5d 5d 5e 5e 5e 5f 5f 5f 60 60 60 61 61 61 62 62 62 63 63 63
00000400 64 64 64 65 65 65 66 66 66 67 67 67 68 68 68 69 69 69 6a 6a 6a 6b 6b 6b 6c 6c 6c 6d 6d 6d
00000440 6e 6e 6e 6f 6f 6f 70 70 70 71 71 71 72 72 72 73 73 73 74 74 74 75 75 75 76 76 76 77 77 77
00000480 78 78 78 78 79 79 79 7a 7a 7a 7b 7b 7b 7c 7c 7c 7d 7d 7d 7e 7e 7e 7f 7f 7f 7f 7f 7f 7f 7f
```

9. SCSI Debugging

Using the SCSI Inquiry example above, add debug options will provide more information:

```

[root@rtppa-rhel001 ~]# spt inquiry logprefix= enable=debug,debug
Opening device /dev/mapper/mpathae, open flags = 044002 (0x4802)...
Device /dev/mapper/mpathae successfully opened, fd = 3
SCSI I/O Structure:
Device Special File ..... /dev/mapper/mpathae
File Descriptor ..... fd: 4
Data Direction ..... dxfer_direction: -3 (SG_DXFER_FROM_DEV)
Control Flags ..... flags: 0
SCSI CDB Status ..... status: 0 (SCSI_GOOD)
SCSI Masked Status ..... masked_status: 0
Command Timeout ..... timeout: 60000 ms (60 seconds)
Command Descriptor Block ..... cdb: 12 0 0 0 ff 0 (Inquiry)
CDB Length ..... cmd_len: 6
I/O Buffer Address ..... dxferp: 0x19e6000
I/O Buffer Length ..... dxfer_len: 255 (0xff)
I/O Data Residual ..... resid: 187 (0xbb)
Request Sense Buffer ..... sbp: 0x19e7000
Request Sense Length ..... mx_sb_len: 255 (0xff)
Request Sense Returned ..... sb_len_wr: 0 (0)
Host Status ..... host_status: 0 (DID_OK)
Driver Status ..... driver_status: 0 (DRIVER_OK)
Messaging Level Data (optional) ..... msg_status: 0

CDB Data Received: (68 bytes)

offset  Data
000000  00 00 05 32 3f 18 10 02 4e 69 6d 62 6c 65 20 20 " 2? Nimble "
000010  53 65 72 76 65 72 20 20 20 20 20 20 20 20 20 20 "Server"
000020  31 2e 30 20 35 2e 31 00 00 00 00 00 00 00 01 00 "1.0 5.1"
000030  00 00 00 00 00 00 00 00 00 00 00 60 09 60 08 c0 " "
000040  03 00 03 20 " "

Inquiry Information:
Length: 68 (0x44)
offset 0 (0x0): 00 00 05 32 3f 18 10 02 4e 69 6d 62 6c 65 20 20
offset 16 (0x10): 53 65 72 76 65 72 20 20 20 20 20 20 20 20 20 20
offset 32 (0x20): 31 2e 30 20 35 2e 31 00 00 00 00 00 00 00 01 00
offset 48 (0x30): 00 00 00 00 00 00 00 00 00 00 00 60 09 60 08 c0
offset 64 (0x40): 03 00 03 20
Peripheral Device Type: 0 (Direct Access Device)
Peripheral Qualifier: 0 (Peripheral Device Connected)
Reserved (byte 1, bits 0:5): 0
Logical Unit Conglomerate: False
Removable Media: False
ANSI Version: 5 (Complies to ANSI INCITS 408-2005, SPC-3)
Response Data Format: 2
Historical Support: True
Normal ACA Support: True
Reserved (byte 3, bits 6:2): 0
Additional Length: 63 (0x3f)
Supports Protection Information: False
Reserved (byte 5, bits 1:2): 0
Third Party Copy Support: True
Target Port Group Support: 0x1 (implicit ALUA)
Obsolete (byte 5, bit 6): 0
Storage Controller Components: False
Obsolete (byte 6, bit 0): 0
Reserved (byte 6, bits 1:2): 0
Obsolete (byte 6, bit 3): 0
Multiple SCSI Ports: True
Vendor Specific (byte 6, bit 5): False
Embedded Enclosure Services: False
Obsolete (byte 6, bit 7): False
Vendor Specific (byte 7, bit 0): False
Command Queuing Support: True
Reserved (byte 7, bit 2): 0
Obsolete (byte 7, bits 3:3): 0
Reserved (byte 7, bit 6): 0
Obsolete (byte 7, bit 7): 0
Vendor Identification: Nimble
Product Identification: Server
Firmware Revision Level: 1.0
Array Software Version: 5.1
Target Type: 1 (Group Scoped)
Synchronous Replication: False
Reserved (byte 47, bits 3:5): 0
Reserved Bytes 48-56: 00 00 00 00 00 00 00 00 00
Vendor Data Length: 11 (0xb)
Vendor Specific Data: 00 00 00 60 09 60 08 c0 03 00 03

Closing device /dev/mapper/mpathae, fd 4...
Closing device /dev/mapper/mpathae, fd 3...
[root@rtppa-rhel001 ~]#

```

Please Note: Some of the bits may appear incorrect, since I updated Inquiry to ANSI SPC-5 spec.

9.1. Decoding SCSI Error Information

These simple commands can be useful to decode SCSI errors when tools have not translated to a message:

```
[root@rtppa-rhel001 ~]# spt show scsi status=0x2
SCSI Status = 0x2 = SCSI_CHECK_CONDITION
[root@rtppa-rhel001 ~]# spt show scsi key=0x2
Sense Key = 0x2 = NOT_READY
[root@rtppa-rhel001 ~]# spt show scsi ascq=0x040B
Sense Code/Qualifier = (0x4, 0xb) = Logical unit not accessible, target port in standby state
[root@rtppa-rhel001 ~]#
```

10. SCSI Information

Similar to the dt tool, reporting SCSI information is available, but you need to enable it via:

```
[root@rtppa-rhel001 Scripts]# spt dsf=/dev/mapper/mpathh cdb=0 enable=scsi
spt (j:1 t:1):
spt (j:1 t:1): SCSI Information:
spt (j:1 t:1):      SCSI Device: /dev/mapper/mpathh
spt (j:1 t:1):      Inquiry information: vid=Nimble, Pid=Server, Rev=1.0
spt (j:1 t:1):      Block Length: 512
spt (j:1 t:1):      Maximum Capacity: 2097152 (1024.000 Mbytes)
spt (j:1 t:1):      Provisioning Management: Thin Provisioned
spt (j:1 t:1):      Device Identifier: 1a6e-a54b-e31e-259f-6c9c-e900-8314-8ac5
spt (j:1 t:1):      Device Serial Number: 1a6ea54be31e259f6c9ce90083148ac5
spt (j:1 t:1):
spt (j:1 t:1): Nimble Information:
spt (j:1 t:1):      Array Software Version: 5.1
spt (j:1 t:1):      Target Type: Group Scoped
spt (j:1 t:1):      Target Name: 56:c9:ce:90:d7:25:73:00
spt (j:1 t:1):      LU Admin Name: test
spt (j:1 t:1):      Client Mgmt IP Address: 10.64.66.101
spt (j:1 t:1):      Protocol Type: FCP
spt (j:1 t:1):      ITN Addresses: 10:00:00:10:9b:40:07:fd,56:c9:ce:90:d7:25:73:05
spt (j:1 t:1):      Synchronous Replication: False
spt (j:1 t:1):
[root@rtppa-rhel001 Scripts]#
```

10.1. SCSI Statistics

Note that spt does **not** display statistics automatically. The reporting of statistics is via the "**emit=**" option, and is a free form that you the user must define. Special format control strings control the information reported, and "**spt help**" shows these under the heading "Emit Status Format Control:" section:

```

Emit Status Format Control:
  %progname = Our program name (spt).
  %thread = The thread number.
  %cdb = The SCSI CDB bytes.
  %dir = The data direction.
  %length = The data length.
  %adsf = The alternate special file.
  %dsf = The device special file.
  %dsf1 = The 2nd device special file.
  %dst = The destination device name.
  %src = The source device name.
  %src[1-2] = The other source devices.
  %srcs = All the source devices.
  %status = The command (IOCTL) status.
  %status_msg = The IOCTL status message.
  %scsi_name = The SCSI opcode name.
  %scsi_status = The SCSI status.
  %scsi_msg = The SCSI message.
  %host_status = The host status.
  %host_msg = The host status message.
  %driver_status = The driver status.
  %driver_msg = The driver status message.
  %sense_code = The sense error code.
  %sense_msg = The sense code message.
  %info_valid = The information valid bit.
  %info_data = The information field data.
  %cspec_data = The cmd spec information data.
  %resid = The residual bytes.
  %blocks = The blocks transferred.
  %capacity = The device capacity (in blocks).
  %device_size = The device block size.
  %iterations = The iterations executed.
  %operations = The operations executed.
  %starting = The starting logical block.
  %ending = The ending logical block.
  %total_blocks = The total blocks transferred.
  %total_operations = The total operations executed.
  %xfer = The bytes transferred. (or %bytes)
  %total_xfer = The total bytes transferred.
  %sense_key = The sense key.
  %key_msg = The sense key message.
  %ill = Illegal length indicator.
  %eom = End of medium.
  %fm = Tape file mark.
  %ascq = The asc/ascq pair.
  %ascq_msg = The asc/ascq message.
  %asc = The additional sense code.
  %asq = The additional sense qualifier.
  %fru = The field replaceable unit code.
  %sense_data = All the sense data.
  %timeout = The command timeout (in ms).
  %deallocated = The deallocated blocks.
  %mapped = The mapped blocks.

```

While parsing the emit string, spt accepts "**default**", which results in the following:

```

[root@rtppa-rhel001 Scripts]# spt dsf=/dev/mapper/mpathh cdb=0 emit=default
spt (j:1 t:1):
spt (j:1 t:1):      Thread: 1
spt (j:1 t:1):      Device Name: /dev/mapper/mpathh
spt (j:1 t:1):      Device Info: Block Length=512, capacity=0
spt (j:1 t:1):      Block Range: 0 - 0
spt (j:1 t:1):      SCSI Name: Test Unit Ready
spt (j:1 t:1):      SCSI CDB: 00 00 00 00 00 00
spt (j:1 t:1):      Data Direction: none
spt (j:1 t:1):      Data Length: 0
spt (j:1 t:1):      Exit Status: 0 = SUCCESS
spt (j:1 t:1):      Host Status: 0 = DID_OK
spt (j:1 t:1):      Driver Status: 0 = DRIVER_OK
spt (j:1 t:1):      SCSI Status: 0 = SCSI_GOOD
spt (j:1 t:1):      Sense Code: 0 = NO CODE
spt (j:1 t:1):      Sense Key: 0 = NO SENSE
spt (j:1 t:1):      asc/asq: (00, 00) = No additional sense information
spt (j:1 t:1):      Bytes Transferred: 0 (data bytes transferred)
spt (j:1 t:1):      Residual: 0 (bytes not transferred)
spt (j:1 t:1):      Iterations: 1
spt (j:1 t:1):      Total Bytes: 0
spt (j:1 t:1):      Total Blocks: 0
spt (j:1 t:1):      Total Operations: 1
spt (j:1 t:1):      Sense Data: 00 00 00 00 00 00 00 00
spt (j:1 t:1):      Elapsed Time: 00m00.00s
spt (j:1 t:1):      Starting Time: Thu May 23 11:39:31 2019
spt (j:1 t:1):      Ending Time: Thu May 23 11:39:31 2019
spt (j:1 t:1):
[root@rtppa-rhel001 Scripts]#

```

11. List Disks Script

A Python 2.7 script was developed around the Linux spt tool, utilizing "*spt show devices*" output plus acquiring additional information.

Key Points:

- expects spt in same directory as *lsdisks* script, or in \$PATH
- options exist to specify devices and/or output format
- this script supports storage enclosures and disk devices
- *lsdisks* talks to spt over a pipe, avoids process overhead, so is *very fast*!
 - single threaded tight now, so could be made faster for large configs

11.1. *lsdisks* Help Text

```
[root@rtppa-rhel001 Scripts]# lsdisks --help
usage: lsdisks [-h] [--debug] [--drives DRIVES] [--exclude EXCLUDE] [--json] [--long] [--noencs] [--noheader] [--power_on_hours]
               [--firmware_version FIRMWARE_VERSION] [--product_name PRODUCT_NAME] [--vendor_name VENDOR_NAME]
               [--serial_number SERIAL_NUMBER] [--sas_address SAS_ADDRESS] [--target_port TARGET_PORT] [--spt_path SPT_PATH]
               [--scsi_paths SCSI_PATHS] [--hostname HOSTNAME] [--password PASSWORD] [--username USERNAME]

optional arguments:
  -h, --help            show this help message and exit
  --debug               The debug flag. (Default: False)
  --drives DRIVES       The drives to display. (Default: [])
  --exclude EXCLUDE     The drives to exclude. (Default: [])
  --json               Enable JSON format. (Default: False)
  --long               Enable long format. (Default: False)
  --noencs             Exclude enclosures. (Default: False)
  --noheader           Exclude headers. (Default: False)
  --power_on_hours      Include power on hours. (Default: True)
  --firmware_version FIRMWARE_VERSION
                        The firmware version. (Default: None)
  --product_name PRODUCT_NAME
                        The product name. (Default: None)
  --vendor_name VENDOR_NAME
                        The vendor name. (Default: None)
  --serial_number SERIAL_NUMBER
                        The serial number. (Default: None)
  --sas_address SAS_ADDRESS
                        The SAS address. (Default: None)
  --target_port TARGET_PORT
                        The target port. (Default: None)
  --spt_path SPT_PATH   The spt tool path. (Default: /usr/local/bin/spt)
  --scsi_paths SCSI_PATHS
                        The spt directory paths. (Default: all)
  --hostname HOSTNAME   The remote host name. (Default: None)
  --password PASSWORD   The password for login. (Default: nimbo1i)
  --username USERNAME   The user name for login. (Default: root)
[root@rtppa-rhel001 Scripts]#
```

11.2. Example Brief Output Format:

SCSI PASS-THROUGH (SPT) TOOL

```
[root@rtppqa-rhel001 ~]# lsdisks
```

Linux Device	SCSI Device	Vendor	Array SW Version	Drive Capacity	Block Length	Serial Number	Sync Repl	Path State	Protocol Type	LUN Admin Name
/dev/dm-0	/dev/dm-1	VMware		83886080	512	<not available>	False	Active		
/dev/sda	/dev/sg0		/dev/nimblestorage/rtpp-afa77-fc-rtppqa-rhel001-vol0-21d83cea103fcacba6c9ce900ed956192	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Standby	FCP	rtp-afa77-fc-rtppqa-rhel001-vol0
/dev/dm-2	/dev/mapper/mpathac	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Standby	FCP	
/dev/sdb	/dev/sg2			20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Standby	FCP	
/dev/sde	/dev/sg5	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Standby	FCP	
/dev/sdh	/dev/sg8	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Standby	FCP	
/dev/sdk	/dev/sg11	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Standby	FCP	
/dev/sdn	/dev/sg14	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Standby	FCP	
/dev/sdq	/dev/sg17	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol0
/dev/sdt	/dev/sg20	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol0
/dev/sdw	/dev/sg23	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol0
/dev/sdz	/dev/sg26	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol0
/dev/sdac	/dev/sg29	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol0
/dev/sdaf	/dev/sg32	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol0
/dev/sdai	/dev/sg35	Nimble	5.1	20480000	512	1d83cea103fcacba6c9ce900ed956192	False	Standby	FCP	
/dev/dm-3	/dev/mapper/mpathad		/dev/nimblestorage/rtpp-afa77-fc-rtppqa-rhel001-vol1-2aa6960b38d0cf0946c9ce900ed956192	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Standby	FCP	
/dev/sdc	/dev/sg3	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Standby	FCP	
/dev/sdf	/dev/sg6	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Standby	FCP	
/dev/sdi	/dev/sg9	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Standby	FCP	
/dev/sdl	/dev/sg12	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Standby	FCP	
/dev/sdo	/dev/sg15	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Standby	FCP	
/dev/sdr	/dev/sg18	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol1
/dev/sdu	/dev/sg21	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol1
/dev/sdx	/dev/sg24	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol1
/dev/sdaa	/dev/sg27	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol1
/dev/sdad	/dev/sg30	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol1
/dev/sdag	/dev/sg33	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol1
/dev/sdaj	/dev/sg36	Nimble	5.1	40960000	512	aa6960b38d0cf0946c9ce900ed956192	False	Standby	FCP	
/dev/dm-4	/dev/mapper/mpathae		/dev/nimblestorage/rtpp-afa77-fc-rtppqa-rhel001-vol2-7b465faa2e1855ac6c9ce900ed956192	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	
/dev/sdd	/dev/sg4	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	
/dev/sdg	/dev/sg7	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	
/dev/sdj	/dev/sg10	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	
/dev/sdm	/dev/sg13	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	
/dev/sdp	/dev/sg16	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	
/dev/sds	/dev/sg19	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdv	/dev/sg22	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdy	/dev/sg25	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdab	/dev/sg28	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdae	/dev/sg31	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdah	/dev/sg34	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdak	/dev/sg37	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	

```
[root@rtppqa-rhel001 ~]#
```

11.3. Example Long Output Format:

```
[root@rtppqa-rhel001 ~]# lsdisks --drives /dev/sdq --long
Device Type: disk
Device Description: Direct Access Device
Linux SCSI Nexus: [3:0:9:0]
Linux Device Name: /dev/sdq
SCSI Device Name: /dev/sg17
Product Identification: Server
Vendor Identification: Nimble
Firmware Version: 1.0
Array Software Version: 5.1
Target Type: 1
Target Type Description: Group Scoped
Synchronous Replication: False
Serial Number: 1d83cea103fcacba6c9ce900ed956192
Device World Wide Name: 0x1d83cea103fcacba6c9ce900ed956192
Drive Capacity: 20480000
Block Length: 512
Target Name: 56:c9:ce:90:ef:ed:ec:00
LUN Admin Name: rtp-afa77-fc-rtppqa-rhel001-vol0
Client Mgmt IP Address: 10.64.66.101
Path State: Active
ITN Addresses: 10:00:00:10:9b:40:07:fd, 56:c9:ce:90:ef:ed:ec:02
Protocol Type: 2
Protocol Description: FCP
[root@rtppqa-rhel001 ~]#
```

11.4. Example JSON Output Format

```
[root@rtppqa-rhel001 ~]# lsdisks --drives /dev/sdq --json
[
  {
    "SAS Address": "",
    "LU Admin Name": "rtp-afa77-fc-rtppqa-rhel001-vol0",
    "ITN Addresses": "10:00:00:10:9b:40:07:fd,56:c9:ce:90:ef:ed:ec:02",
    "Array Software Version": "5.1",
    "Device Type": "disk",
    "Device Type Description": "Direct Access Device",
    "Enclosure Slot": "",
    "Enclosure Device": "",
    "Vendor Identification": "Nimble",
    "Revision Level": "1.0",
    "Protocol Type": 2,
    "Block Length": 512,
    "Current Temperature": "",
    "Device world wide Name": "0x1d83cea103fcacba6c9ce900ed956192",
    "Serial Number": "1d83cea103fcacba6c9ce900ed956192",
    "Target Type Description": "Group Scoped",
    "Linux SCSI Nexus": "[3:0:9:0]",
    "Client Mgmt IP Address": "10.64.66.101",
    "Product Identification": "Server",
    "Drive Capacity": 20480000,
    "SCSI Device Name": "/dev/sg17",
    "Synchronous Replication": false,
    "Firmware Version": "1.0",
    "Path State": "Active",
    "DMMP Device Name": [],
    "First Device Entry": true,
    "Target Name": "56:c9:ce:90:ef:ed:ec:00",
    "Linux Device Name": "/dev/sdq",
    "Power On Hours": "",
    "Slot Description": "",
    "Protocol Description": "FCP",
    "Target Type": 1
  }
]
[root@rtppqa-rhel001 ~]#
```

11.5. Remote Example

Internally, *lsdisks* is talking to *spt* via pipes, so assuming an ssh connection can be established, one can query remote systems:

```
[romiller@rtm-rtp-rhel17 ~]$ lsdisks --host rtppqa-rhel001 --serial 7b465faa2e1855ac6c9ce900ed956192
```

Linux Device	SCSI Device	Vendor	Array SW Version	Drive Capacity	Block Length	Serial Number	Sync Rep	Path State	Protocol Type	LUN Admin Name
/dev/dm-4	/dev/mapper/mpathae	/dev/nimblestorage	rtp-afa77-fc-rtppqa-rhel001-vol2	27b465faa2e1855ac6c9ce900ed956192						
/dev/sdd	/dev/sg4	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	
/dev/sdg	/dev/sg7	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	
/dev/sdj	/dev/sg10	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	
/dev/sdm	/dev/sg13	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	
/dev/sdp	/dev/sg16	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	
/dev/sds	/dev/sg19	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdv	/dev/sg22	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdy	/dev/sg25	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdab	/dev/sg28	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdae	/dev/sg31	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdah	/dev/sg34	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Active	FCP	rtp-afa77-fc-rtppqa-rhel001-vol2
/dev/sdak	/dev/sg37	Nimble	5.1	61440000	512	7b465faa2e1855ac6c9ce900ed956192	False	Standby	FCP	

```
[romiller@rtm-rtp-rhel17 ~]$
```